

# METODOLOGÍA ROTATIVA ITERATIVA: DESARROLLO DE VIDEOJUEGOS C#

Programación de Videojuegos I — UTN FRT | Cátedra: Profesor Matías Chocobar

Fase 1: Arquitectura de Software POO I y POO II (10 Proyectos de Videojuegos Reales)

## ⚠ [ENTREGA OBLIGATORIA GITHUB]

**FECHA LÍMITE DE ENTREGA: VIERNES 16 DE OCTUBRE DE 2026 HASTA LAS 23:50 HS.**

**No se recibirán entregas por correo electrónico ni ningún otro medio que no sea Pull Request en GitHub.**

**Estructura obligatoria de carpetas: Todo el código y reporte debe estar contenido dentro de la carpeta: 'grupo/' (Ejemplo: grupo/Grupo\_01/).**

## 1. METODOLOGÍA ROTATIVA Y FLUJO COLABORATIVO GITHUB (LÍDER Y EQUIPO)

La cátedra implementa la metodología de rotación de código profesional (Code Handover & Code Review):

- Cada uno de los 10 grupos recibe asignado un videojuego específico por el docente (los alumnos no eligen tema).
- En la Semana 1 (actual), cada grupo programa la arquitectura lógica en C# con mínimo 4 clases integrando POO 1 y POO 2.
- En las semanas siguientes, el docente ROTARÁ los repositorios entre grupos: cada equipo recibirá el proyecto de otro grupo, deberá auditar el código recibido, solucionar bugs, completar funciones faltantes e integrarle la nueva unidad (físicas 2D en Unity).

FLUJO TÉCNICO DE GITHUB PASO A PASO:

### 1. Acción del Líder del Grupo:

- Ingresa al repositorio oficial del Profesor Matías Chocobar:

<https://github.com/ChocobarMatias/Progrmacion-1-TUDPV-Comision-1>

- Realiza un FORK a su cuenta personal de GitHub (ejemplo: <https://github.com/LiderGrupo/Progrmacion-1-TUDPV-Comision-1>).
- En Settings de su Fork, añade a sus compañeros de equipo como Colaboradores (Collaborators).

### 2. Acción de los Integrantes del Grupo:

- NO clonan al docente: Clonan el FORK del Líder a sus computadoras locales mediante GitHub Desktop.
- Cada integrante crea obligatoriamente una rama con su nombre y legajo: branch 'dev-NombreApellido-Legajo'.
- Desarrolla su aporte en Visual Studio dentro de la carpeta 'grupo/Grupo\_XX/'.
- Realiza Commit y Push de su rama hacia el repositorio del Líder.

### 3. Integración Final y Pull Request al Docente:

- El Líder revisa los aportes de sus compañeros y realiza el MERGE de las ramas a la rama 'main' de su Fork.
- Finalmente, el Líder abre un PULL REQUEST (PR) desde su Fork hacia el repositorio del Profesor Matías Chocobar.
- Título obligatorio del PR: [ENTREGA FASE 1] Grupo XX - Tema: NombreDelJuego.

### 4. REPORTE TÉCNICO OBLIGATORIO (reporte.md dentro de la carpeta del grupo):

Todo grupo debe incluir un archivo 'reporte.md' con las siguientes secciones:

- a) Lista de integrantes (Nombre, Apellido y Legajo).
- b) Diagnóstico de inicio (Cómo se concibió la arquitectura inicial).
- c) Registro de cambios y módulos implementados.

- d) Solución de problemas y bugs prevenidos mediante encapsulamiento.
- e) Guía de prueba de la consola (cómo probar los métodos en Main).

## 2. REQUERIMIENTOS TÉCNICOS OBLIGATORIOS (POO I + POO II)

Cada uno de los 10 proyectos debe cumplir estrictamente con los siguientes estándares de arquitectura C#:

1. Mínimo 4 Clases Interconectadas: Una clase base abstracta o protegida, mínimo dos clases derivadas (hijas), y una clase controladora/gestora de sistemas o inventario.
2. Encapsulamiento y Protección de Datos: Uso riguroso de modificadores 'private' y 'protected'. No se admiten atributos críticos 'public' sin validación.
3. Propiedades (get / set): Implementación de propiedades automáticas y propiedades con validación mediante la palabra clave 'value'.
4. Constructores Parametrizados y 'this': Desambiguación explícita de campos mediante 'this' y delegación a clases padres mediante ': base(...)'.  
5. Herencia y Polimorfismo Real: Declaración de métodos 'virtual' en las clases base y sobrescritura especializada 'override' en las clases hijas.
6. Colección de Objetos en Main(): Demostración de arrays o listas polimórficas (ej. Entidad[] lista) iteradas con bucles donde se ejecutan acciones conjuntas.

## 3. ENUNCIADOS DE LOS 10 TEMAS ASIGNADOS (JUEGOS COMERCIALES)

### TEMA 1: SUPER MARIO BROS (Nintendo)

Objetivo:

Modelar el sistema de personajes y power-ups de Mario.

Estructura de 4 Clases Obligatoria:

- Clase Base 'PersonajeMario': Atributos protegidos (nombre, vidas, monedas). Propiedad 'Vidas' con validación (si llega a 0, 'GameOver'). Constructor base. Método virtual 'Saltar()' y método 'RecibirDanio()'.  
• Clase Hija 'Mario': Hereda de PersonajeMario. Atributo privado 'estadoTransformacion' (Normal, Super, Fuego). Sobrescribe 'Saltar()' para alcanzar 2 bloques de altura.  
• Clase Hija 'Luigi': Hereda de PersonajeMario. Sobrescribe 'Saltar()' con mayor altura pero menor tracción.  
• Clase 'BloqueInterrogacion': Atributo 'abierto', contiene un método 'Golpear(PersonajeMario pj)' que entrega monedas o power-up validando que no se golpee dos veces.  
• Main(): Crear a Mario y Luigi en un array 'PersonajeMario[] hermanos', hacerlos golpear bloques y saltar polimórficamente.

### TEMA 2: THE LEGEND OF ZELDA (Nintendo)

Objetivo:

Modelar el sistema de combate y equipo de Link en mazmorras.

Estructura de 4 Clases Obligatoria:

- Clase Base 'ItemZelda': Propiedad 'Nombre', atributo protegido 'durabilidad'. Constructor. Método virtual 'Usar()'.  
• Clase Hija 'EspadaMaestra': Hereda de ItemZelda. Atributo 'danioSagrado'. Sobrescribe 'Usar()' lanzando rayos de energía si Link tiene salud máxima.  
• Clase Hija 'ArcoFuerza': Hereda de ItemZelda. Atributo 'flechasRestantes'. Sobrescribe 'Usar()' gastando flechas con validación.  
• Clase 'Link': Atributos privados 'nombre', 'corazones', y un inventario 'ItemZelda[] alforja' de 2 slots. Método 'Equipar()' y método 'AccionarItem(int slot)'.  
• Main(): Link interactúa con la alforja ejecutando sus armas polimórficamente.

### TEMA 3: POKÉMON (Game Freak)

Objetivo:

Modelar el sistema de combate por turnos de criaturas Pokémon.

Estructura de 4 Clases Obligatoria:

- Clase Base 'Pokemon': Atributos protegidos (apodo, salud, nivel). Propiedad 'Salud' controlada [0, 100]. Constructor con 'this'. Método virtual 'Atacar()'.  
Construtor con 'this'. Método virtual 'Atacar()'.
- Clase Hija 'PokemonFuego' (ej. Charmander): Sobrescribe 'Atacar()' ejecutando 'Lanzallamas' con cálculo de daño de fuego.
- Clase Hija 'PokemonAgua' (ej. Squirtle): Sobrescribe 'Atacar()' ejecutando 'Pistola de Agua'.
- Clase 'Entrenador': Atributo 'nombre' y arreglo 'Pokemon[] equipo' de hasta 3 criaturas. Métodos 'Capturar()' y 'ComandarAtaqueEquipo()'.
- Main(): El entrenador ordena atacar a todo su equipo recorriendo el array polimórficamente.

### TEMA 4: STREET FIGHTER (Capcom)

Objetivo:

Modelar el sistema de luchadores de torneo arcade y técnicas especiales.

Estructura de 4 Clases Obligatoria:

- Clase Base 'Luchador': Atributos protegidos (nombre, barraSalud, barraKi). Constructor con validación. Método virtual 'EjecutarEspecial()'.
- Clase Hija 'Ryu': Sobrescribe 'EjecutarEspecial()' lanzando 'Hadoken' gastando 25 de Ki.
- Clase Hija 'ChunLi': Sobrescribe 'EjecutarEspecial()' ejecutando 'Kikoken' con mayor velocidad.
- Clase 'RingTorneo': Administra dos luchadores 'luchador1' y 'luchador2'. Método 'IniciarRound()' y método 'DeclararGanador()' evaluando barras de salud.
- Main(): Instanciar los luchadores, ejecutar especiales y verificar ganador.

### TEMA 5: DOOM (id Software)

Objetivo:

Modelar el arsenal de armamento pesado y demonios del infierno.

Estructura de 4 Clases Obligatoria:

- Clase Base 'ArmaDoom': Propiedades 'Nombre', atributo protegido 'municion'. Método virtual 'Disparar(Demonio objetivo)'.
- Clase Hija 'SuperShotgun': Gasta 2 cartuchos por tiro y duplica el daño a quemarropa.
- Clase Hija 'BFG9000': Gasta celda de plasma de 40 unidades y genera daño masivo en área.
- Clase 'Demonio': Atributos privados 'tipo', 'salud'. Propiedad 'Salud' que detecta si el demonio muere ('Glory Kill disponible').
- Main(): El Doom Slayer recorre su arsenal en un array disparando a los demonios.

### TEMA 6: MINECRAFT (Mojang)

Objetivo:

Modelar la recolección de bloques y desgaste de herramientas de minería.

Estructura de 4 Clases Obligatoria:

- Clase Base 'Herramienta': Atributo protegido 'material', durabilidad privada con propiedad. Método virtual 'PicarBloque(Bloque b)'.
- Clase Hija 'PicoHierro': Mayor velocidad de extracción y desgaste de 1 de durabilidad por golpe.
- Clase Hija 'PicoDiamante': Capacidad de minar obsidiana y desgaste mínimo.
- Clase 'Bloque': Propiedad 'Tipo' (Piedra, Diamante), 'dureza' (int). Método 'Romper()'.
- Main(): Demostrar el minado de distintos bloques con desgaste de herramientas.

## TEMA 7: DARK SOULS (FromSoftware)

Objetivo:

Modelar el sistema de combate exigente, consumo de stamina y hogueras.

Estructura de 4 Clases Obligatoria:

- Clase Base 'GuerreroHueco': Atributos protegidos (nombre, salud, stamina). Método virtual 'Esquivar()' y método 'Atacar()'.
- Clase Hija 'CaballeroNegro': Gran escudo; sobrescribe 'Esquivar()' con un rodaje pesado que consume 40 de stamina.
- Clase Hija 'AsesinoSombra': Armadura ligera; sobrescribe 'Esquivar()' con un 'Fast-roll' ágil que consume 15 de stamina.
- Clase 'Hoguera': Método 'Descansar(GuerreroHueco g)' que restablece la salud y stamina pero revive a los enemigos.
- Main(): Simular combate, esquivas y descanso en la hoguera.

## TEMA 8: RESIDENTIAL EVIL (Capcom)

Objetivo:

Modelar el inventario de supervivencia, gestión de munición y zombies.

Estructura de 4 Clases Obligatoria:

- Clase Base 'ConsumibleSurvival': Atributo protegido 'nombre'. Método virtual 'Consumir(Superviviente s)'.
- Clase Hija 'HierbaVerde': Restaura 40 HP y cura estados alterados.
- Clase Hija 'SprayPrimerosAuxilios': Restaura el 100% de salud pero tiene un solo uso.
- Clase 'Superviviente': Atributos privados 'nombre', 'salud', 'envenenado' (bool). Arreglo de inventario con espacios limitados (4 slots).
- Main(): Gestionar el uso de suministros en el inventario limitado.

## TEMA 9: HOLLOW KNIGHT (Team Cherry)

Objetivo:

Modelar el sistema de combate de insectos, el agujón y la acumulación de Alma.

Estructura de 4 Clases Obligatoria:

- Clase Base 'CriaturaHallownest': Atributos protegidos 'nombre', 'salud'. Constructor base. Método virtual 'ReaccionarGolpe()'.
- Clase Hija 'ElCaballero' (Protagonista): Atributo privado 'reservaAlma' [0, 99]. Método 'ConcentrarAlma()' para curarse y sobrescribe ataque.
- Clase Hija 'JefeHusk': Enemigo acorazado; sobrescribe 'ReaccionarGolpe()' liberando infección.
- Clase 'Amuleto': Modifica atributos del Caballero (ej. aumenta el largo del agujón o daño).
- Main(): Enfrentamiento entre El Caballero y el Jefe usando alma y amuletos.

## TEMA 10: PAC-MAN (Bandai Namco)

Objetivo:

Modelar el laberinto, estados de fantasmas y consumo de frutas.

Estructura de 4 Clases Obligatoria:

- Clase Base 'Fantasma': Atributos protegidos (nombre, color, estadoVulnerable bool). Método virtual 'Move() y 'SerDevorado()'.
- Clase Hija 'Blinky' (Rojo): Modo persecución directa; sobrescribe 'Move()' aumentando velocidad.
- Clase Hija 'Inky' (Azul): Emboscada; sobrescribe 'Move()' según la posición de Pac-Man.
- Clase 'PacMan': Atributos 'vidas', 'puntos'. Método 'ComerPildoraPoder(Fantasma[] fantasmas)' que cambia el estado polimórficamente a vulnerable.
- Main(): Pac-Man come una píldora de poder y devora a los fantasmas en bucle.